

Product API – Infrastructure Deployment (Hw5)

This project implements the Product API defined in the provided OpenAPI specification and deploys it to AWS using Docker, ECR, ECS (Fargate), and Terraform.

The infrastructure is **fully automated** using Terraform and can be deployed on any machine with minimal setup.

Repository Structure

```
Hw5/
├── main.go           # Product API server implementation
├── go.mod
├── go.sum
├── Dockerfile        # Docker image for local testing
└── terraform-config/
    ├── src/          # Server code used for ECS deployment
    │   ├── main.go
    │   ├── go.mod
    │   ├── go.sum
    │   └── Dockerfile
    └── terraform/
        ├── provider.tf
        ├── variables.tf
        ├── main.tf
        ├── output.tf
        └── modules/
```

How to Deploy on a Different Machine

Anyone in the group can deploy this system by following these steps.

Step 1: Install Requirements

Install:

- **Docker Desktop**
- **Terraform**
- **AWS CLI**
- **Git**

Verify installation:

```
docker version
terraform -version
aws --version
git --version
```

Step 2: Clone Repository

```
git clone <your-repo-url>
cd Hw5/terraform-config/terraform
```

Step 3: Configure AWS Credentials

```
aws configure
```

Enter:

- AWS Access Key
- AWS Secret Key
- Region (e.g., `us-east-1`)
- Output format: `json`

Verify credentials:

```
aws sts get-caller-identity
```

Step 4: Initialize Terraform

```
terraform init
```

Step 5: Deploy Infrastructure

```
terraform apply
```

Type:

```
yes
```

Terraform will automatically:

- ✓ Create ECR repository
- ✓ Build Docker image
- ✓ Push image to ECR
- ✓ Create ECS cluster
- ✓ Create ECS task definition
- ✓ Launch ECS Fargate service
- ✓ Create security group (port 8080 open)
- ✓ Create CloudWatch log group

Step 6: Retrieve Public IP

After deployment:

```
terraform output
```

Then retrieve the ECS task public IP using AWS CLI or AWS Console.

Your API will be accessible at:

```
http://<public-ip>:8080
```

API Endpoints

Create Product

POST /products

Example Request:

```
POST http://<public-ip>:8080/products
```

Body:

```
{
  "product_id": 1,
  "sku": "ABC-123-XYZ",
  "manufacturer": "Acme Corporation",
  "category_id": 456,
  "weight": 1250,
  "some_other_id": 789
}
```

Possible Responses:

- 201 Created
 - 400 Bad Request
 - 409 Conflict
-

Get Product

GET /products/{productId}

Example:

```
GET http://<public-ip>:8080/products/1
```

Possible Responses:

- 200 OK
 - 400 Bad Request
 - 404 Not Found
-

Update Product Details

POST /products/{productId}/details

Example:

```
POST http://<public-ip>:8080/products/1/details
```

Body:

```
{
  "product_id": 1,
  "sku": "ABC-123-UPDATED",
  "manufacturer": "Acme Corporation",
  "category_id": 456,
  "weight": 1500,
  "some_other_id": 789
}
```

Possible Responses:

- 204 No Content
- 400 Bad Request
- 404 Not Found

Server Response Examples

The following response codes were verified using **Postman**:

- ✓ **201 Created** – Product successfully created
- ✓ **200 OK** – Product successfully retrieved
- ✓ **204 No Content** – Product successfully updated
- ✓ **400 Bad Request** – Invalid input
- ✓ **404 Not Found** – Non-existent product
- ✓ **409 Conflict** – Duplicate product

The first screenshot shows the execution of the `terraform init` command. The output indicates that Terraform is reusing previous versions of the `kreuzwerker/docker` and `hashicorp/aws` providers from the dependency lock file. It then proceeds to install the `hashicorp/aws` v6.7.0 provider and the `kreuzwerker/docker` v2.25.0 provider. A message states that partner and community providers are signed by their developers and provides a link for more information on provider signing. The terminal concludes with the message: "Terraform has been successfully initialized!" and instructions on how to use `terraform plan` to see required changes.

The second screenshot shows the execution of `terraform plan`. It prompts for confirmation to perform the actions, which are accepted by entering 'yes'. The plan shows that two resources will be added: `docker_image.app` and `docker_registry_image.app`. The `docker_image.app` resource is created successfully after 325 seconds. The `docker_registry_image.app` resource is also created successfully after 2 seconds. The plan concludes with the message: "Apply complete! Resources: 2 added, 0 changed, 0 destroyed." and displays the output for the `ecs_cluster_name` and `ecs_service_name` attributes.

POST

http://34.229.160.90:8080/products

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Sci

1

{

2

"product_id": 1,

3

"sku": "ABC-123-XYZ",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1250,

7

}

Body

Cookies

Headers (3)

Test Results

201 Created

618 ms

237 B

Sav

{}

JSON

Preview

Visualize

1

{

2

"product_id": 1,

3

"sku": "ABC-123-XYZ",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1250,

7

"some_other_id": 789

8

}

GET

http://34.229.160.90:8080/products/1

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

200 OK

19 ms

232 B

Sa

{}

JSON

Preview

Visualize

1

{

2

"product_id": 1,

3

"sku": "ABC-123-XYZ",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1250,

7

"some_other_id": 789

8

}

Product API cloud Testing / Update Product

Save

POST

http://34.229.160.90:8080/products/1/details

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Sch

1 {

2 "product_id": 1,

3 "sku": "ABC-123-UPDATED",

4 "manufacturer": "Acme Corporation",

5 "category_id": 456,

6 "weight": 1500,

Body

Cookies

Headers (1)

Test Results

204 No Content

20 ms

64 B

Save

Raw

Preview

Visualize

1

GEThttp://34.229.160.90:8080/products/1

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

200 OK

19 ms

236 B

Sav

{}

JSON

Preview

Visualize

1

{

2

"product_id": 1,

3

"sku": "ABC-123-UPDATED",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1500,

7

"some_other_id": 789

8

}

GEThttp://34.229.160.90:8080/products/2

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

404 Not Found

181 ms

205 B

Sav

{}

JSON

Preview

Debug with AI

1

{

2

"error": "NOT_FOUND",

3

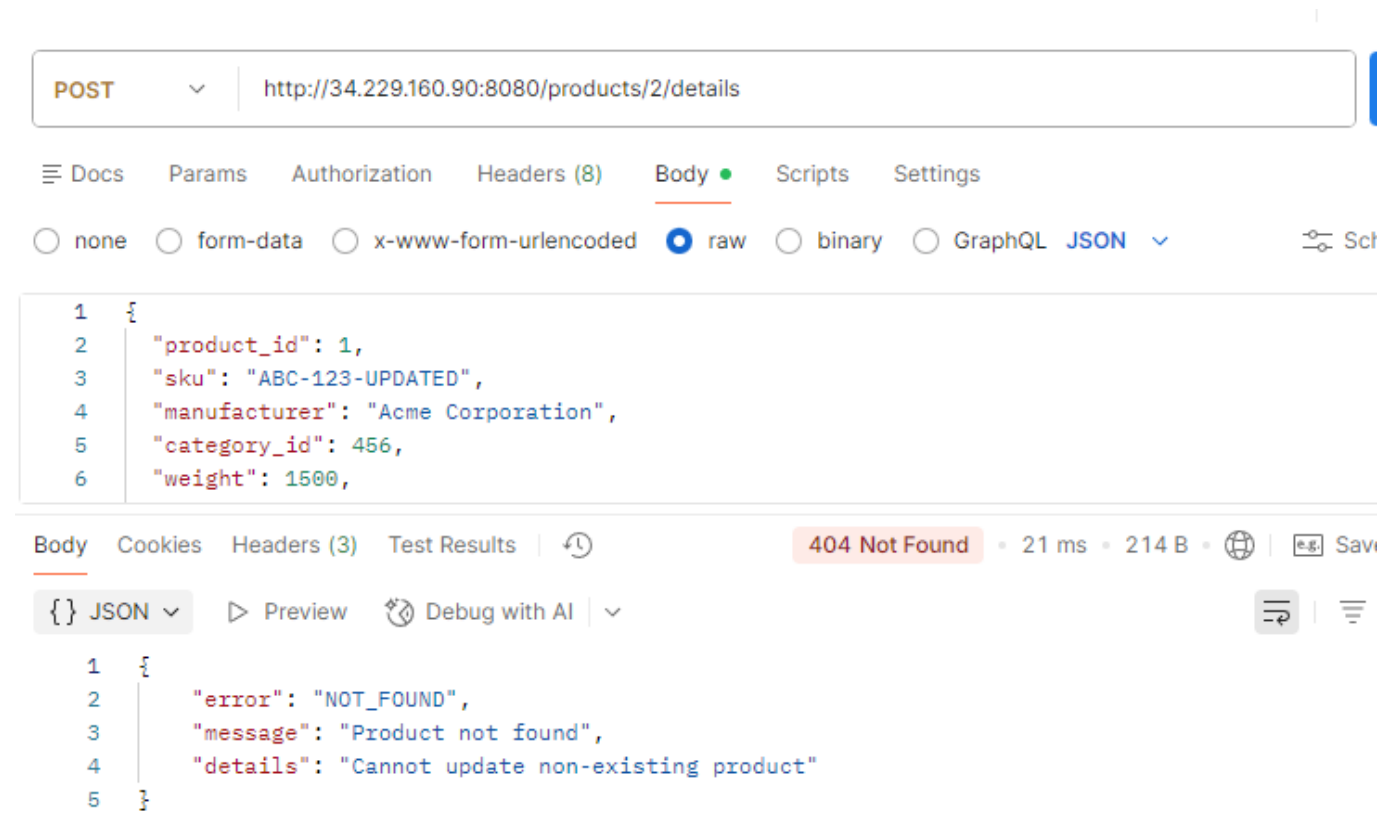
"message": "Product not found",

4

"details": "Product ID does not exist"

5

}



Data Storage

Product data is stored **in memory** using:

```
map[int]Product
```

No database is used as per assignment requirements. Data is volatile and will reset on container restart.

Infrastructure Components

AWS Services Used

- **ECR (Elastic Container Registry)** – Docker image storage
- **ECS (Elastic Container Service)** – Container orchestration
- **Fargate** – Serverless compute for containers
- **CloudWatch** – Logging and monitoring
- **Security Groups** – Network access control

Infrastructure as Code

- **Terraform** – Automated infrastructure provisioning
- **Docker** – Containerization

Cleanup

To destroy all AWS resources:

```
terraform destroy
```

Type:

```
yes
```

This will remove all created resources and prevent ongoing charges.

Notes

- This is a homework assignment (Hw5) implementing cloud infrastructure deployment
- The API follows RESTful conventions
- All infrastructure is provisioned via Terraform for reproducibility
- The application runs on ECS Fargate (serverless containers)
- Logs are available in CloudWatch for debugging